

EXP. NO: 3

EDA-Data Cleaning

AIM:

To clean and preprocess the dataset by handling missing values, removing duplicates, converting data types, maintaining consistency, and normalizing numerical data.

PROCEDURE:

1. Import the required libraries such as Pandas, NumPy, Seaborn, Matplotlib, and Scikit-learn.
2. Load the dataset using `pd.read_csv()`.
3. Display the dataset information and identify missing values.
4. Visualize missing values using a heatmap.
5. Fill missing values using mode for categorical columns and median for numerical columns.
6. Remove duplicate records from the dataset.
7. Convert data into appropriate data types.
8. Ensure consistency in categorical values by removing spaces and standardizing capitalization.
9. Normalize numerical features using Min-Max Scaling.
10. Standardize the Credit_History column using StandardScaler.
11. Display the cleaned dataset and its final information

PROGRAM:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler
df = pd.read_csv('/content/test_Y3wMUE5_7gLdaTN.csv')
print("Initial Data Overview:")
df.info()
print("\nMissing Values:")
print(df.isnull().sum())
sns.heatmap(df.isnull(), cbar=False, cmap="Blues")
plt.title("Missing Value Heatmap")
plt.show()
for col in ['Gender', 'Married', 'Dependents', 'Self_Employed']:
    df[col] = df[col].fillna(df[col].mode()[0])
df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].median())
df['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(
    df['Loan_Amount_Term'].mode()[0])
df['Credit_History'] = df['Credit_History'].fillna(
    df['Credit_History'].mode()[0])
initial_rows = df.shape[0]
df.drop_duplicates(inplace=True)
print(f"\nRemoved {initial_rows - df.shape[0]} duplicate rows.")
df['Dependents'] = (df['Dependents'].replace('3+', 3).fillna(0).astype(int))
```

```

for col in ['Gender', 'Married', 'Education',
            'Self_Employed', 'Property_Area']:
    df[col] = df[col].str.strip().str.capitalize()
min_max_scaler = MinMaxScaler()
scale_cols = [ 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']
df[scale_cols] = min_max_scaler.fit_transform(df[scale_cols])
scaler = StandardScaler()
df[['Credit_History']] = scaler.fit_transform(
    df[['Credit_History']])
print("\nCleared Data Summary:")
df.info()
print("\nFirst 5 rows of cleaned data:")
print(df.head())

```

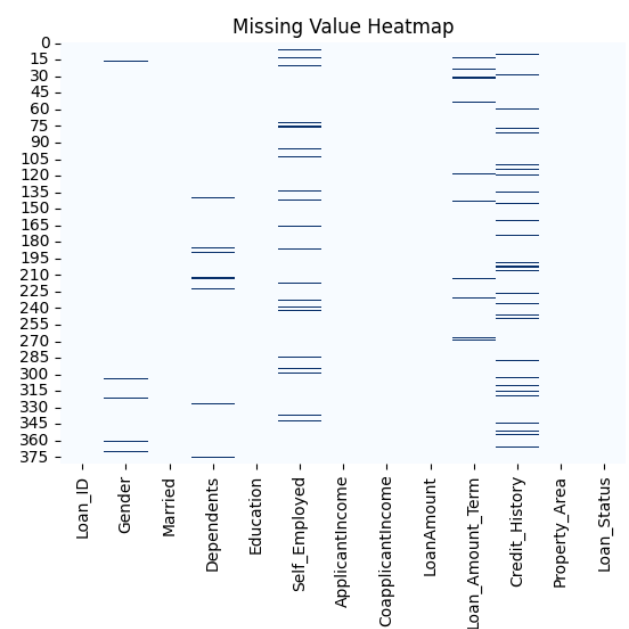
OUTPUT:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

	Company	Contact	Country
0	Alfreds Futterkiste	Maria Anders	Germany
1	Centro comercial Moctezuma	Francisco Chang	Mexico
2	Ernst Handel	Roland Mendel	Austria
3	Island Trading	Helen Bennett	UK
4	Laughing Bacchus Winecellars	Yoshi Tannamuri	Canada

```
Initial Data Overview:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 381 entries, 0 to 380
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Loan_ID                381 non-null   object
1   Gender                 376 non-null   object
2   Married                381 non-null   object
3   Dependents             373 non-null   object
4   Education              381 non-null   object
5   Self_Employed          360 non-null   object
6   ApplicantIncome        381 non-null   int64
7   CoapplicantIncome      381 non-null   float64
8   LoanAmount             381 non-null   float64
9   Loan_Amount_Term       370 non-null   float64
10  Credit_History         351 non-null   float64
11  Property_Area          381 non-null   object
12  Loan_Status            381 non-null   object
dtypes: float64(4), int64(1), object(8)
memory usage: 38.8+ KB
None
```

```
Missing Values in Each Column:
Loan_ID                0
Gender                 5
Married                0
Dependents             8
Education              0
Self_Employed         21
ApplicantIncome        0
CoapplicantIncome      0
LoanAmount             0
Loan_Amount_Term       11
Credit_History        30
Property_Area          0
Loan_Status            0
dtype: int64
```



```
Cleaned Data Summary:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 381 entries, 0 to 380
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Loan_ID                381 non-null   object
1   Gender                 381 non-null   object
2   Married                381 non-null   object
3   Dependents             381 non-null   int64
4   Education              381 non-null   object
5   Self_Employed          381 non-null   object
6   ApplicantIncome        381 non-null   float64
7   CoapplicantIncome      381 non-null   float64
8   LoanAmount             381 non-null   float64
9   Loan_Amount_Term       381 non-null   float64
10  Credit_History         381 non-null   float64
11  Property_Area          381 non-null   object
12  Loan_Status            381 non-null   object
dtypes: float64(5), int64(1), object(7)
memory usage: 38.8+ KB
None
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001003	Male	Yes	1	Graduate	No	
1	LP001005	Male	Yes	0	Graduate	Yes	
2	LP001006	Male	Yes	0	Not graduate	No	
3	LP001008	Male	No	0	Graduate	No	
4	LP001013	Male	Yes	0	Not graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	0.464043	0.044567	0.843972	360.0	
1	0.298336	0.000000	0.404255	360.0	
2	0.254684	0.069687	0.787234	360.0	
3	0.612373	0.000000	0.936170	360.0	
4	0.228515	0.044803	0.609929	360.0	

	Credit_History	Property_Area	Loan_Status
0	0.419435	Rural	N
1	0.419435	Urban	Y
2	0.419435	Urban	Y
3	0.419435	Urban	Y
4	0.419435	Urban	Y

RESULT:

Thus, the given program was successfully executed to clean the dataset by handling missing values, removing duplicates, converting data types, ensuring consistency, and normalizing the data.